Федеральное государственное образовательное бюджетное
учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики» (ФГОБУ ВО «СибГУТИ»)

Основы систем мобильной связи

Лабораторная работа 4

Выполнил: Клишин Владиммир Евгеньевич ИКС - 431

Новосибирск 2026

Цель работы:
Получить представление о том, какие существуют псевдослучайные двоичные последовательности, какими корреляционными свойствами они обладают и как используются для синхронизации приемников и передатчиков в сетях мобильной связи.

1) Написать программу на языке C/C++ для генерации последовательности Голда.

Код:

```
#include <iostream>

using namespace std;

int main() {
int gl = 0;
int x_b = 0b01010; //10
int y_b = 0b10001; //17
int mask = 0b11111;

for (int i = 0; i < 31; i++) {
int feedback_y = ((y_b >> 1) & 1) ^ (y_b & 1);
int feedback_x = ((x_b >> 3) & 1) ^ (x_b & 1);
int output = (x_b & 1) ^ (y_b & 1);

gl = (gl << 1) | output;

x_b = ((x_b >> 1) | (feedback_x << 4)) & mask;
y_b = ((y_b >> 1) | (feedback_y << 4)) & mask;
}

return 0; }
```

2) Сделать поэлементный циклический сдвиг последовательности и посчитать автокорреляцию исходной последовательности и сдвинутой. Сформировать таблицу с битовыми значениями последовательностей, в последнем столбце которой будет вычисленное значение автокорреляции

Код:

```
#include <iostream>

using namespace std;

double get_kor(int gl, int sdvig) {
int matches = 0;
int mismatches = 0;

for (int i = 0; i < 31; i++) {
int bit_original = (gl >> i) & 1;
int bit_shifted = (gl >> ((i + sdvig) % 31)) & 1;

if (bit_original == bit_shifted) {
matches++;
} else {
mismatches++;
}
}

return (matches - mismatches) / 31.0;
}

void print_table(int gl) {
cout << "Сдвиг | 31 бит последовательности | Автокорреляция" << endl;
cout << "-----------------------------------------------------------" << endl;

for (int s = 0; s <= 31; s++) {
cout << " " << s << " | ";

for (int i = 0; i < 31; i++) {
int bit = (gl >> ((i + s) % 31)) & 1;
cout << bit;
}
cout << " | " << get_kor(gl, s) << endl;
}
}

int main() {
int gl = 0;
int x_b = 0b01010; //10
int y_b = 0b10001; //17
int mask = 0b11111;

for (int i = 0; i < 31; i++) {
int feedback_y = ((y_b >> 1) & 1) ^ (y_b & 1);
int feedback_x = ((x_b >> 3) & 1) ^ (x_b & 1);
int output = (x_b & 1) ^ (y_b & 1);
```

```
gl = (gl << 1) | output;

x_b = ((x_b >> 1) | (feedback_x << 4)) & mask;
y_b = ((y_b >> 1) | (feedback_y << 4)) & mask;
}


print_table(gl);

return 0;
}
```

Вывод:

```
Сдвиг | 31 бит последовательности          | Автокорреляция
-------------------------------------------------------------
 0   | 0101111010001100011001111011011 | 1
 1   | 1011110100011000110011110110110 | -0.0322581
 2   | 0111101000110001100111101101101 | -0.16129
 3   | 1111010001100011001111011011010 | -0.0322581
 4   | 1110100011000110011110110110101 | -0.0322581
 5   | 1101000110001100111101101101011 | 0.354839
 6   | 1010001100011001111011011010111 | -0.0322581
 7   | 0100011000110011111011011010111 | -0.16129
 8   | 1000110001100111101101101011110 | -0.0322581
 9   | 0001100011001111011011010111101 | 0.225806
 10  | 0011000110011110110110101111010 | -0.0322581
 11  | 0110001100111101101101011110100 | -0.16129
 12  | 1100011001111011011010111101000 | -0.0322581
 13  | 1000110011110110110101111010001 | 0.0967742
 14  | 0001100111101101101011110100011 | 0.0967742
 15  | 0011001111011011010111101000110 | -0.16129
 16  | 0110011110110110101111010001100 | -0.16129
 17  | 1100111101101101011110100011000 | 0.0967742
 18  | 1001111011011010111101000110001 | 0.0967742
 19  | 0011110110110101111010001100011 | -0.0322581
 20  | 0111101101101011110100011000110 | -0.16129
 21  | 1111011011010111101000110001100 | -0.0322581
 22  | 1110110110101111010001100011001 | 0.225806
 23  | 1101101101011110100011000110011 | -0.0322581
 24  | 1011011010111101000110001100111 | -0.16129
 25  | 0110110101111010001100011001111 | -0.0322581
 26  | 1101101011110100011000110011110 | 0.354839
 27  | 1011010111101000110001100111101 | -0.0322581
 28  | 0110101111010001100011001111011 | -0.0322581
 29  | 1101011110100011000110011110110 | -0.16129
 30  | 1010111101000110001100111101101 | -0.0322581
 31  | 0101111010001100011001111011011 | 1
```

3)Сформировать еще одну последовательность Голда, такую что x=x+1, а y= y-5

Код:

```
#include <iostream>

using namespace std;

double get_kor(int gl, int sdvig) {
int matches = 0;
int mismatches = 0;

for (int i = 0; i < 31; i++) {
int bit_original = (gl >> i) & 1;
int bit_shifted = (gl >> ((i + sdvig) % 31)) & 1;

if (bit_original == bit_shifted) {
matches++;
} else {
mismatches++;
}
}

return (matches - mismatches) / 31.0;
}

void print_table(int gl) {
cout << "Сдвиг | 31 бит последовательности | Автокорреляция" << endl;
cout << "-----------------------------------------------------------" << endl;

for (int s = 0; s <= 31; s++) {
cout << " " << s << " | ";

for (int i = 0; i < 31; i++) {
int bit = (gl >> ((i + s) % 31)) & 1;
cout << bit;
}
cout << " | " << get_kor(gl, s) << endl;
}
}



int main() {
int gl = 0;
int gl_n = 0;
int x_b = 0b01010; //10
int x_bn = 0b1011; //11
int y_b = 0b10001; //17
int y_bn = 0b1100; //12
int mask = 0b11111;
```

```
for (int i = 0; i < 31; i++) {
int feedback_y = ((y_b >> 1) & 1) ^ (y_b & 1);
int feedback_x = ((x_b >> 3) & 1) ^ (x_b & 1);
int output = (x_b & 1) ^ (y_b & 1);

gl = (gl << 1) | output;

x_b = ((x_b >> 1) | (feedback_x << 4)) & mask;
y_b = ((y_b >> 1) | (feedback_y << 4)) & mask;
}

for (int i = 0; i < 31; i++) {
int feedback_y = ((y_bn >> 1) & 1) ^ (y_bn & 1);
int feedback_x = ((x_bn >> 3) & 1) ^ (x_bn & 1);
int output = (x_bn & 1) ^ (y_bn & 1);

gl_n = (gl_n << 1) | output;

x_bn = ((x_bn >> 1) | (feedback_x << 4)) & mask;
y_bn = ((y_bn >> 1) | (feedback_y << 4)) & mask;
}

print_table(gl);


return 0;
}
```

4)Вычислить значение взаимной корреляции исходной и новой последовательностей и вывести в терминал

Код:

```
#include <iostream>

using namespace std;

double get_kor(int gl, int sdvig) {
int matches = 0;
int mismatches = 0;

for (int i = 0; i < 31; i++) {
int bit_original = (gl >> i) & 1;
int bit_shifted = (gl >> ((i + sdvig) % 31)) & 1;

if (bit_original == bit_shifted) {
matches++;
} else {
```

```
mismatches++;
}
}

return (matches - mismatches) / 31.0;
}

double get_cross_kor(int gl, int gl_n, int sdvig) {
int matches = 0;
int mismatches = 0;

for (int i = 0; i < 31; i++) {
int bit1 = (gl >> i) & 1;
int bit2 = (gl_n >> ((i + sdvig) % 31)) & 1;

if (bit1 == bit2) matches++;
else mismatches++;
}
return (matches - mismatches) / 31.0;
}

void print_table(int gl) {
cout << "Сдвиг | 31 бит последовательности | Автокорреляция" << endl;
cout << "-----------------------------------------------------------" << endl;

for (int s = 0; s <= 31; s++) {
cout << " " << s << " | ";

for (int i = 0; i < 31; i++) {
int bit = (gl >> ((i + s) % 31)) & 1;
cout << bit;
}
cout << " | " << get_kor(gl, s) << endl;
}
}

void print_cross_table(int gl, int gl_n) {

cout << "Сдвиг | 31 бит последовательности (gl_n) | Взаим. корр." << endl;
cout << "-----------------------------------------------------" << endl;

for (int s = 0; s <= 31; s++) {
cout << " " << s << " | ";
for (int i = 0; i < 31; i++) {
cout << ((gl_n >> ((i + s) % 31)) & 1);
}
cout << " | " << get_cross_kor(gl, gl_n, s) << endl;
}
}
```

```
int main() {
int gl = 0;
int gl_n = 0;
int x_b = 0b01010; //10
int x_bn = 0b1011; //11
int y_b = 0b10001; //17
int y_bn = 0b1100; //12
int mask = 0b11111;

for (int i = 0; i < 31; i++) {
int feedback_y = ((y_b >> 1) & 1) ^ (y_b & 1);
int feedback_x = ((x_b >> 3) & 1) ^ (x_b & 1);
int output = (x_b & 1) ^ (y_b & 1);

gl = (gl << 1) | output;

x_b = ((x_b >> 1) | (feedback_x << 4)) & mask;
y_b = ((y_b >> 1) | (feedback_y << 4)) & mask;
}

for (int i = 0; i < 31; i++) {
int feedback_y = ((y_bn >> 1) & 1) ^ (y_bn & 1);
int feedback_x = ((x_bn >> 3) & 1) ^ (x_bn & 1);
int output = (x_bn & 1) ^ (y_bn & 1);

gl_n = (gl_n << 1) | output;

x_bn = ((x_bn >> 1) | (feedback_x << 4)) & mask;
y_bn = ((y_bn >> 1) | (feedback_y << 4)) & mask;
}

print_table(gl);

cout << endl;

print_cross_table(gl, gl_n);

return 0;
}
```

Вывод:

```
Сдвиг | 31 бит последовательности (gl_n) | Взаим. корр.
-------------------------------------------------
  0   | 01101101001001111010101000001111 | -0.16129
  1   | 11011010010011110101010000011110 | 0.0967742
  2   | 10110100100111101010100000111101 | -0.0322581
  3   | 01101001001111010101000001110111 | -0.0322581
  4   | 11010010011110101010000011101110 | -0.16129
  5   | 10100100111101010100000111011101 | -0.16129
```

6	01001001111010101000001110110 11	0.225806
7	10010011110101010000011101101 10	-0.0322581
8	00100111101010100000111011011 01	-0.0322581
9	01001111010101000001110110110 10	0.225806
10	10011110101010000011101101101 00	0.0967742
11	00111101010100000111011011010 01	0.0967742
12	01111010101000001110110110100 10	0.354839
13	11110101010000011101101101001 00	-0.419355
14	11101010100000111011011010010 01	0.0967742
15	11010101000001110110110100100 11	0.225806
16	10101010000011101101101001001 11	-0.16129
17	01010100000111011011010010011 11	0.225806
18	10101000001110110110100100111 10	-0.16129
19	01010000011101101101001001111 01	-0.16129
20	10100000111011011010010011110 10	-0.0322581
21	01000001110110110100100111101 01	-0.16129
22	10000011101101101001001111010 10	-0.16129
23	00000111011011010010011110101 01	0.225806
24	00001110110110100100111101010 10	0.225806
25	00011101101101001001111010101 00	-0.0322581
26	00111011011010010011110101010 00	-0.16129
27	01110110110100100111101010000 00	0.0967742
28	11101101101001001111010101000 00	-0.0322581
29	11011011010010011110101010000 01	0.0967742
30	10110110100100111101010100000 11	-0.0322581
31	01101101001001111010101000001 11	-0.16129

5)Проделать шаги 1-4 в Matlab. Используя функции xcorr() и autocorr() для вычисления соответствующих корреляций. Вывести на график в Matlab функцию автокорреляции в зависимости от величины задержки (lag).

```
x_b  = 10;
y_b  = 17;

x_bn = 11;
y_bn = 12;

mask = 31;

gl   = zeros(1, 31);
gl_n = zeros(1, 31);

for i = 1:31

   % gl
   fb_x = bitxor(bitget(x_b, 4), bitget(x_b, 1));
   fb_y = bitxor(bitget(y_b, 2), bitget(y_b, 1));
```

```
    gl(i) = bitxor(bitget(x_b, 1), bitget(y_b, 1));

    x_b = bitshift(x_b, -1) + fb_x * 16;
    y_b = bitshift(y_b, -1) + fb_y * 16;

    % gl_n
    fb_x = bitxor(bitget(x_bn, 4), bitget(x_bn, 1));
    fb_y = bitxor(bitget(y_bn, 2), bitget(y_bn, 1));

    gl_n(i) = bitxor(bitget(x_bn, 1), bitget(y_bn, 1));

    x_bn = bitshift(x_bn, -1) + fb_x * 16;
    y_bn = bitshift(y_bn, -1) + fb_y * 16;

end

gl   = fliplr(gl);
gl_n = fliplr(gl_n);

% Вывод последовательностей
disp('Первая последовательность:');
disp(gl);

disp('Вторая последовательность:');
disp(gl_n);

gl_bip   = 2 * gl - 1;
gl_n_bip = 2 * gl_n - 1;

% Авто
[acf_xcorr, lags_acf] = xcorr(gl_bip, 'coeff');
acf_auto = autocorr(gl_bip', 'NumLags', 30);

% Взаимная
[ccf_xcorr, lags_ccf] = xcorr(gl_bip, gl_n_bip, 'coeff');

disp('Автокорреляция (xcorr):');
disp([lags_acf(lags_acf >= 0)', acf_xcorr(lags_acf >= 0)']);

disp('Автокорреляция (autocorr):');
disp([(0:30)', acf_auto(:)]);

disp('Взаимная корреляция (xcorr):');
disp([lags_ccf(lags_ccf >= 0)', ccf_xcorr(lags_ccf >= 0)']);

figure;

stem(lags_acf(lags_acf >= 0), acf_xcorr(lags_acf >= 0), 'filled');
hold on;

stem(0:30, acf_auto, 'x');
```

```
title('Автокорреляция последовательности gl');
xlabel('Задержка (lag)');
ylabel('Коэффициент корреляции');

legend('xcorr', 'autocorr');
grid on;
xticks(0:5:30);
ylim([-0.5 1.2]);

hold off;
```

Вывод:

```
Автокорреляция (xcorr):
         0    1.0000
    1.0000    0.0000
    2.0000   -0.1613
    3.0000   -0.0645
    4.0000    0.0323
    5.0000    0.2581
    6.0000   -0.0968
    7.0000   -0.1290
    8.0000   -0.0968
    9.0000    0.2581
   10.0000   -0.0968
   11.0000   -0.1290
   12.0000   -0.0323
   13.0000   -0.0645
   14.0000    0.0323
   15.0000   -0.1290
   16.0000   -0.0323
   17.0000    0.0645
   18.0000    0.1613
   19.0000   -0.0000
   20.0000   -0.0323
   21.0000    0.0645
   22.0000   -0.0323
   23.0000    0.0645
   24.0000   -0.0323
   25.0000    0.0645
   26.0000    0.0968
   27.0000   -0.0645
   28.0000    0.0323
   29.0000    0.0000
   30.0000   -0.0323

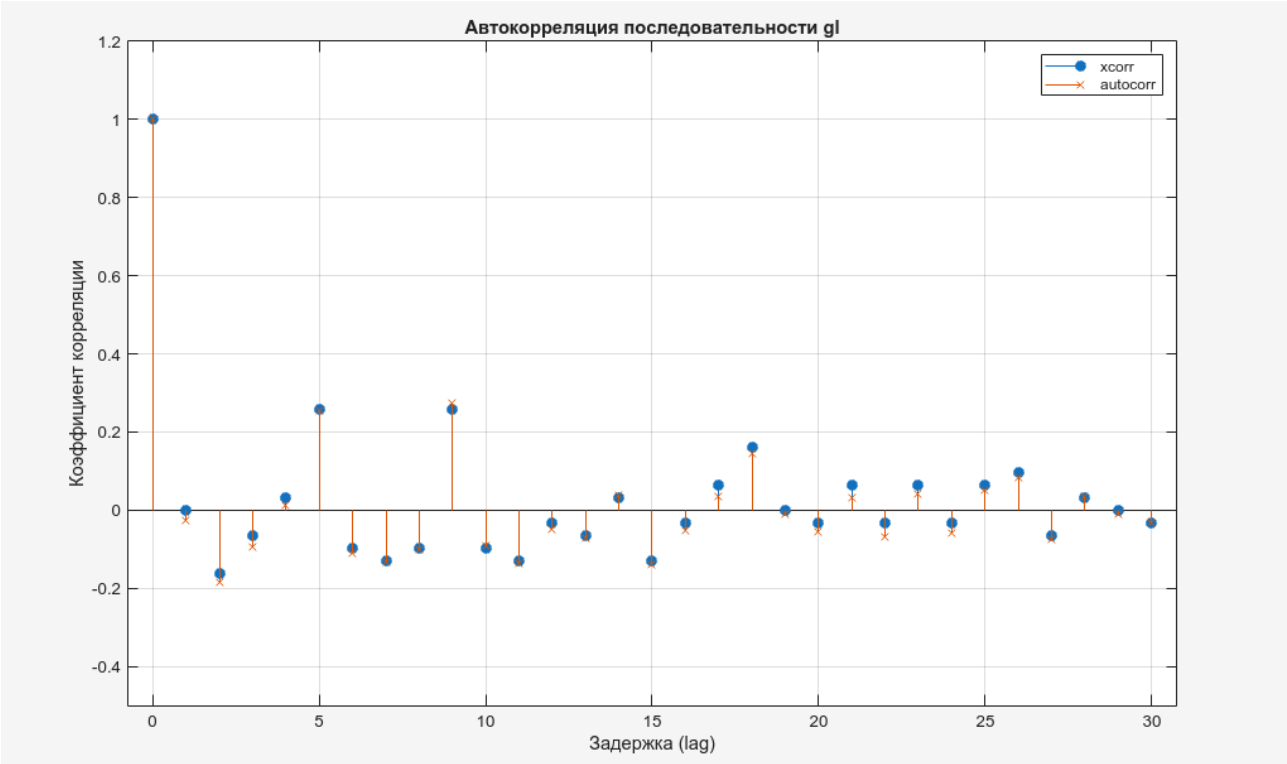
Автокорреляция (autocorr):
         0    1.0000
    1.0000   -0.0276
    2.0000   -0.1833
```

```
   3.0000   -0.0955
   4.0000    0.0136
   5.0000    0.2553
   6.0000   -0.1099
   7.0000   -0.1332
   8.0000   -0.1009
   9.0000    0.2732
  10.0000   -0.0919
  11.0000   -0.1366
  12.0000   -0.0488
  13.0000   -0.0721
  14.0000    0.0371
  15.0000   -0.1401
  16.0000   -0.0522
  17.0000    0.0356
  18.0000    0.1447
  19.0000   -0.0110
  20.0000   -0.0557
  21.0000    0.0321
  22.0000   -0.0681
  23.0000    0.0411
  24.0000   -0.0591
  25.0000    0.0500
  26.0000    0.0823
  27.0000   -0.0735
  28.0000    0.0357
  29.0000   -0.0090
  30.0000   -0.0323

Взаимная корреляция (xcorr):
         0   -0.1613
    1.0000   -0.0000
    2.0000    0.0968
    3.0000   -0.0000
    4.0000    0.0323
    5.0000   -0.1935
    6.0000   -0.1613
    7.0000    0.1290
    8.0000    0.2258
    9.0000   -0.0645
   10.0000   -0.0968
   11.0000    0.1935
   12.0000   -0.0968
   13.0000   -0.0000
   14.0000    0.0323
   15.0000   -0.1935
   16.0000    0.2258
   17.0000    0.1290
   18.0000   -0.2903
   19.0000    0.1290
   20.0000    0.0968
   21.0000   -0.1290
```

22.0000	0.1613
23.0000	-0.0645
24.0000	-0.0968
25.0000	0.1935
26.0000	-0.0323
27.0000	-0.0645
28.0000	0.0968
29.0000	-0.0000
30.0000	-0.0323

График:



Вывод:
В ходе лабораторной работы были сформированы две псевдослучайные последовательности Голда. С помощью MATLAB были рассчитаны их автокорреляционные и взаимокорреляционные функции, а также построен график автокорреляции. В результате были изучены корреляционные свойства последовательностей и освоены функции xcorr() и autocorr()

Ссылка на Git:

Ответы на контрольные вопросы:

1)Какие модели распространения сигналов используются для расчета радиопокрытия сетей мобильной связи?

2)Какие основные составляющие бюджета восходящего (UL) и нисходящего (DL) каналов в радиосетях?

3)Чем отличается чувствительность приемника базовой станции E и пользовательского терминала UE?

4)Что такое тепловой шум и как он определяется?

5)Что ограничивает радиус соты мобильных сетей в нисходящем и восходящем каналах?

6)Из чего состоят потери сигнала в антенно-фидерном тракте базовой станции BS?

1)
Используются: модель свободного пространства (FSPM), UMiNLOS (для микро/фемтосот и помещений), COST 231 Hata (Окумура-Хата) для макросот, Walfish-Ikegami для плотной городской застройки.

2)
Мощность передатчика (TxPower), потери в фидере (FeederLoss), усиление антенны БС (AntGainBS), выигрыш MIMO (MIMOGain), запасы на интерференцию (IM) и проникновение сквозь стены (PenetrationM), чувствительность приёмника (RxSens).
Для UL: передатчик – UE, приёмник – BS. Для DL: передатчик – BS, приёмник – UE.

3)
Чувствительность рассчитывается как RxSens = NoiseFigure + ThermalNoise + RequiredSINR.
У BS и UE разные коэффициенты шума (в работе 2.4 дБ и 6 дБ), разные полосы частот (10 МГц для UL и 20 МГц для DL) и разные требуемые SINR (4 дБ и 2 дБ). Поэтому чувствительности неодинаковы.

4)
Это минимальный уровень шума приёмника, определяемый по формуле:
ThermalNoise = -174 + 10·log10(BW),
где BW – полоса частот в Гц, а -174 дБм – уровень шума на 1 Гц при 20 °C.

5)
В DL радиус ограничен мощностью передатчика BS и чувствительностью UE.
В UL – мощностью передатчика UE и чувствительностью BS.
Итоговый радиус берётся по меньшему из двух значений (обычно UL).

6)
Состоят из потерь в фидере (~2 дБ), потерь в МШУ (~0.4 дБ) и потерь в джампере (~0.5 дБ). Если фидер отсутствует (передатчик рядом с антенной), учитываются только потери в джампере (0.5 дБ).